

Backend Engineer

1. Category - Technology

The modern "Full-Stack Backend" suite for 2026.

1. **Language:** Python 3.10+ (utilising Structural Pattern Matching and Strict Typing)
2. **Framework:** FastAPI (Asynchronous)
3. **Primary DB:** PostgreSQL 16+
4. **ORM:** SQLAlchemy 2.0 (Async)
5. **In-Memory DB:** Redis (for Caching & Rate Limiting)
6. **Data Validation:** Pydantic v2 (Strict Mode)
7. **Task Queue:** Celery + Flower (for Monitoring)
8. **Migrations:** Alembic
9. **Security & Auth:** PyJWT + Passlib (OAuth2 with JWT & RBAC)
10. **Testing:** Pytest + Faker + Testcontainers (to test against real DBs)
11. **Load Testing:** Locust
12. **DevOps & Linting:** Docker, Compose, and Ruff (the 2026 standard for linting/formatting)
13. **Observability:** OpenTelemetry + Prometheus/Grafana (Distributed Tracing)
14. **Deployment:** AWS (ECS/Lambda) using Terraform (Infrastructure as Code)

2. Backend Work

These 12 steps now cover the full lifecycle of a "Senior-grade" project.

1. **Foundation (Base Backend):** Clean Architecture (Repository Pattern), Async DB Setup, API Versioning and Ruff integration.
2. **Identity System (Auth):** Secure JWT login, Password hashing, and Role-Based Access Control (RBAC).
3. **Flash Sale Engine (Core):** Distributed Locking with Redis, handling Race Conditions, and ACID transactions.
4. **Idempotency & Middleware:** Implementation of X-Idempotency-Key for safe retries on financial/critical actions.
5. **Advanced Rate Limiting:** Sliding-window algorithms using Redis to prevent API abuse.
6. **Background Workers:** Complex Celery workflows (Chains/Groups) for non-blocking email and report generation.
7. **Event & Webhook System:** Reliable delivery with retry logic and "Signature Verification" for security.
8. **Resilience Patterns:** Implementing Circuit Breakers and Retries using the Tenacity library.
9. **The "Observe" Layer:** Structured Logging (JSON) and Distributed Tracing (tracking requests across API → Worker).
10. **Security Audit:** Implementing OWASP Top 10 fixes, CORS, and Secret Management (AWS Secrets Manager).
11. **Optimization:** Database indexing strategies, Redis caching layers, and Profiling slow endpoints.
12. **System Polish & IaC:** Graceful Shutdown and Writing Terraform scripts for AWS and preparing the "System Design" documentation for interviews.

AI Engineer

1. Python

The goal: Move from writing scripts to building production-grade software.

Fundamentals (0-1 Year):

1. **Basic Syntax:** Variables, Data Types, Type Casting.
2. **Control Flow:** Conditionals (if/elif/else), Loops (for, while).
3. **Data Structures:** Lists, Tuples, Sets, Dictionaries.
4. **Functions:** Built-in functions, Custom functions, Lambda expressions.
5. **Error Handling:** Try/Except blocks, Exception types.
6. **OOP Basics:** Classes, Objects, Inheritance, Methods, Dunder methods.
7. **Environment:** Pip, PyPI, virtualenv/pyenv.
8. **Basic Packages:** NumPy, Pandas, Matplotlib.

Professional Mastery (1-2 Years):

1. **Advanced Python:** Decorators, Iterators, Generators, Context Managers (with).
 2. **Concurrency:** Threading, Multiprocessing, Asynchrony (asyncio), GIL.
 3. **Strict Typing:** typing module, Mypy, Pydantic (Data validation).
 4. **DSA:** Arrays, Linked Lists, Hash Tables, Heaps, Stacks, Queues, Binary Search Trees, Recursion, Sorting Algorithms.
 5. **Testing:** Pytest, unittest, Mocking.
 6. **Package Management:** Poetry, Conda, pyproject.toml.
 7. **Frameworks:** FastAPI (Async), Flask, or Django.
 8. **Code Quality:** Ruff, Black, Isort, Sphinx (Documentation).
-

2. SQL

The goal: Move from simple queries to database optimization.

Fundamentals (0-1 Year):

1. **Basics:** RDBMS concepts, SQL vs NoSQL.
2. **DDL:** CREATE, ALTER, DROP, TRUNCATE.
3. **DML:** SELECT, INSERT, UPDATE, DELETE.
4. **Filtering:** WHERE, AND/OR, IN, BETWEEN, LIKE.
5. **Joins:** INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN.
6. **Aggregates:** SUM, COUNT, AVG, MIN, MAX → GROUP BY → HAVING.
7. **Constraints:** Primary Key, Foreign Key, Unique, Not Null, Check.

Professional Mastery (1-2 Years):

1. **Advanced Querying:** Common Table Expressions (CTEs), Recursive Queries, Subqueries (Nested & Correlated).
2. **Window Functions:** ROW_NUMBER, RANK, DENSE_RANK, LEAD, LAG.
3. **Database Objects:** Views, Stored Procedures, User-Defined Functions.
4. **Optimization:** Indexing (B-Tree, Hash), Execution Plans (EXPLAIN), Query Tuning.
5. **Transactions:** ACID properties, BEGIN, COMMIT, ROLLBACK, Isolation Levels.
6. **Advanced Logic:** CASE statements, COALESCE, NULLIF.
7. **Vector Search:** pgvector (Essential for AI roles).

3. Machine Learning

The goal: Move from "fitting models" to "evaluating and deploying" them.

Fundamentals (0-1 Year):

1. **Math Foundation:** Linear Algebra (Matrices, Vectors), Calculus (Derivatives), Probability & Statistics (Distributions, CLT).
2. **Types of ML:** Supervised (Classification, Regression) vs. Unsupervised (Clustering, Dimensionality Reduction).
3. **Algorithms:** Linear Regression, Logistic Regression, Decision Trees, Random Forest, K-Nearest Neighbors (KNN), K-Means Clustering.
4. **Preprocessing:** Data Cleaning, Feature Scaling (Normalization/Standardization), One-Hot Encoding.
5. **Basic Evaluation:** Accuracy, Mean Absolute Error (MAE), Mean Squared Error (MSE).

Professional Mastery (1-2 Years):

1. **Advanced Algorithms:** Gradient Boosting (XGBoost, LightGBM, CatBoost), SVM, PCA (Principal Component Analysis).
 2. **Rigorous Evaluation:** Precision, Recall, F1-Score, ROC-AUC, Confusion Matrix, Log Loss.
 3. **Validation:** K-Fold Cross Validation, LOOCV, Train/Test/Validation splits.
 4. **Time Series:** ARIMA, Forecasting, Seasonality.
 5. **MLOps:** Model Versioning (DVC), Experiment Tracking (MLflow), Model Deployment (CI/CD for ML).
 6. **Explainability:** SHAP, LIME (Understanding why a model made a decision).
-

4. Deep Learning

The goal: Move from "black box" models to "architecture optimization."

Fundamentals (0-1 Year):

1. **Neural Network Basics:** Perceptrons, MLP, Forward and Back-Propagation.
2. **Activation Functions:** ReLU, Sigmoid, Tanh, Softmax.
3. **Loss Functions:** Cross-Entropy, MSE.
4. **Libraries:** PyTorch, TensorFlow, Keras.
5. **Architectures:** CNNs (Convolution, Pooling, Padding), RNNs, LSTMs, GRUs.

Professional Mastery (1-2 Years):

1. **Advanced Architectures:** Transformers (Self-Attention, Multi-head Attention), Autoencoders, GANs.
 2. **Training Techniques:** Batch Normalization, Dropout, Learning Rate Schedulers, Early Stopping.
 3. **Transfer Learning:** Fine-tuning pre-trained models (ResNet, BERT), Freezing layers.
 4. **Optimization:** Quantization, Pruning, Knowledge Distillation (Making models smaller/faster).
 5. **Computer Vision/NLP:** Image Segmentation, Tokenization, Lemmatization, Word Embeddings.
-

5. Artificial Intelligence (Modern GenAI)

The goal: Mastering the "Integration" and "Agentic" era of AI (2026 standard).

Fundamentals (0-1 Year):

1. **LLM Basics:** Inference, Training, Tokens, Context Window, Temperature.
2. **Prompts:** Basic Prompt Engineering, System Prompts.
3. **API Integration:** OpenAI API, HuggingFace Hub, Anthropic API.
4. **Open Source:** Using Ollama or Local LLMs.

Professional Mastery (1-2 Years):

1. **RAG (Retrieval Augmented Generation):** Chunking strategies, Embedding models, Vector Databases (Pinecone, Chroma, Weaviate).
 2. **Advanced RAG:** Re-ranking, Query Expansion, HyDE.
 3. **AI Agents:** Tool Use (Function Calling), ReAct prompting, AI Agents (LangChain, LlamaIndex).
 4. **AI Safety:** Prompt Injection, Bias/Fairness, Moderation APIs.
 5. **Multimodal AI:** Vision-Language Models (GPT-4o), Text-to-Speech, Speech-to-Text (Whisper).
-

6. Data Science

The goal: Move from "making charts" to "driving business decisions."

Fundamentals (0-1 Year):

1. **EDA:** Exploratory Data Analysis using Pandas, Seaborn, Matplotlib.
2. **Data Understanding:** Handling missing values, outliers, and data types.
3. **Basic Stats:** Mean, Median, Mode, Standard Deviation, Variance.
4. **Visualization:** Creating Bar charts, Histograms, Scatter plots.

Professional Mastery (1-2 Years):

1. **Hypothesis Testing:** p-values, t-tests, ANOVA, Null Hypothesis.
2. **Experimental Design:** A/B Testing, Sample Size calculation, Minimum Detectable Effect (MDE).
3. **Econometrics:** Regression analysis, Time-series forecasting, Causal Inference.
4. **Metric Design:** Defining KPIs, Ratio Metrics, Delta Method in Analytics.
5. **Storytelling:** Translating technical findings into business insights for stakeholders.

AI Engineer (Jarvis Architect)

Phase 1: The Foundation (Coding & Data)

1. Python: The Nervous System

- **Basics:** Variables, Constants, Types, Type Casting.
- **Logic:** if/elif/else, Logical (and, or, not) & Comparison Operators.
- **Loops:** For, While, break, continue, pass.
- **Structures:** Lists (Slicing), Tuples, Sets, Dictionaries.
- **Functions:** *args, **kwargs, Lambda, Scope.
- **OOP:** Classes, __init__, Inheritance, Dunder methods (__str__, __call__).
- **Advanced:** Decorators, Generators, Context Managers, **Asyncio** (Crucial).
- **Environment:** uv (Modern replacement for pip), Virtualenvs.

2. SQL: The Memory Bank

- **Basics:** CRUD (Select, Insert, Update, Delete).
- **Joins:** Inner, Left, Right, Full.
- **Analytics:** Group By, Having, Aggregates.
- **Advanced:** CTEs, Window Functions, **pgvector** (Storing AI vectors).
- **Migrations:** Alembic (Managing DB changes).

3. Version Control & DevOps Basics

- **Git:** commit, branch, merge, rebase.
- **GitHub:** PRs, Actions (CI/CD).
- **Containerization:** Docker & Docker Compose (To run Jarvis anywhere).

Phase 2: The Brain (Math & Models)

4. Machine Learning (ML)

- **Math:** Linear Algebra, Calculus, Probability.
- **Algos:** Regression, Random Forest, XGBoost, K-Means.
- **Evaluation:** Precision, Recall, F1, ROC-AUC.

5. Deep Learning (DL)

- **Neural Nets:** Backpropagation, Optimizers (Adam), Loss Functions.
- **Architecture:** CNNs, RNNs, LSTMs.
- **Frameworks:** PyTorch (Industry standard).

Phase 3: The Intelligence (Generative AI)

6. LLM Model Interaction

- **Concepts:** Tokens, Context Window, Temperature.
- **Prompt Engineering:** Few-shot, Chain-of-Thought, System Prompts.
- **Validation:** Pydantic (To ensure AI output follows a strict structure).
- **Local AI:** Ollama, LM Studio.

7. Deep NLP & Transformers

- **Attention:** Self-Attention, Multi-head Attention.
- **Fine-Tuning:** LoRA, QLoRA (Training on custom data).

Phase 4: The Jarvis System (Agents & Integration)

8. RAG: Long-term Memory

- **Embeddings:** Converting text to Vectors.
- **Vector DBs:** Pinecone, ChromaDB, or Milvus.
- **Advanced RAG:** Chunking, Re-ranking, Hybrid Search.

9. AI Agents: Taking Action

- **Logic:** ReAct prompting, Function Calling.
- **Orchestration:** LangChain, LangGraph, or CrewAI.
- **System Tools:** Celery & Redis (For handling long AI tasks in the background).

10. Connectivity & Senses

- **MCP:** Model Context Protocol (Connecting to your local PC/Smart Home).
- **STT:** OpenAI Whisper (Hearing).
- **TTS:** ElevenLabs (Speaking).

Phase 5: The Professional Layer (Production)

11. LLMOps & Monitoring

- **Observability:** Langfuse or LangSmith (Tracing every AI "thought").
- **Monitoring:** Sentry (Error tracking) & Prometheus/Grafana.
- **Deployment:** FastAPI, Cloud VPS (AWS/DigitalOcean), SSL/Caddy.

12. Data Science & Strategy

- **EDA:** Pandas, Matplotlib, Seaborn.
- **Evals:** Building "Grade-books" to test if your AI is getting smarter.

Final Milestone: The Jarvis Project

1. **Voice Interaction:** Whisper + ElevenLabs.
 2. **Private Memory:** RAG over your personal local files.
 3. **Autonomous Actions:** Agent that can use **MCP** to check your calendar and email you.
 4. **Production Ready:** Wrapped in **Docker**, monitored with **Langfuse**, and served via **FastAPI**.
-

Titan AI Engineer

(AI Engineer + Backend Combo)

Phase 1: The Foundation (Core Logic & Data)

Goal: Building the nervous system and memory of your application.

1. Python: The Master Language

- **Basics:** Variables, Constants, Dynamic Typing, Type Casting.
- **Logic:** if/elif/else, Logical (and, or, not) & Comparison Operators.
- **Loops:** For, While, break, continue, pass.
- **Data Structures:** Lists (Slicing/Methods), Tuples (Immutability), Sets, Dictionaries (Hash maps).
- **Functions:** *args, **kwargs, Lambda, Scope.
- **Strict Typing:** Type Hinting (Pydantic/MyPy) → Crucial for preventing runtime data-type attacks.
- **OOP:** Classes, Objects, __init__, Inheritance, Polymorphism, Encapsulation, Dunder methods (__str__, __call__).
- **Advanced:** Decorators, Generators (yield), Context Managers (with), **Asyncio** (Asynchronous programming for high-speed APIs).
- **Secure Environment:** uv (Modern package manager), Virtualenvs, pyproject.toml and Secret Management (using .env and .gitignore to prevent API key leaks).

2. SQL & Vector Data: The Memory Bank

- **Basics:** RDBMS concepts, Tables, Rows, Columns.
- **DML/DDDL:** SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, DROP.
- **Relational Logic:** Joins (Inner, Left, Right, Full), Primary/Foreign Keys, Constraints.
- **Analytics:** GROUP BY, HAVING, Aggregates (COUNT, SUM, AVG).
- **Advanced SQL:** CTEs (Common Table Expressions), Window Functions (Rank, Lead, Lag).
- **AI Integration:** pgvector (Storing and searching AI vectors inside PostgreSQL).
- **Data Security:** Principle of Least Privilege (PoLP) → Creating specific DB users with limited permissions (e.g., no DROP TABLE rights for the app user).
- **Migrations:** Alembic (Version control for our database schema).

3. DevOps & Version Control: The Infrastructure

- **Git:** init, add, commit, branch, merge, rebase, handling Conflicts.
- **GitHub:** PRs, Issues, **GitHub Actions** (CI/CD pipelines).
- **Containerization:** **Docker & Docker Compose** (Packaging AI models and backend together), Focus on building Non-Root Containers (creating a limited user inside Docker to prevent host-level attacks).
- **Linux Mastery:** Ubuntu CLI, Shell Scripting, SSH, and Permission management (chmod/chown).

Phase 2: The Brain (Intelligence & Processing)

Goal: Understanding how AI thinks before building the system around it.

4. Machine Learning & Data Science

- **Math:** Linear Algebra, Calculus, Probability, Statistics (Mean, Median, p-values).
- **EDA:** Pandas, Matplotlib, Seaborn (Exploratory Data Analysis).
- **Algorithms:** Regression, Random Forest, XGBoost, K-Means Clustering.
- **Evaluation:** Precision, Recall, F1, ROC-AUC, A/B Testing logic.

5. Deep Learning & Transformers

- **Neural Nets:** Backpropagation, Activation Functions (ReLU, Softmax), Optimizers (Adam).
- **Architecture:** CNNs (Vision), LSTMs (Sequence), **Transformers** (Attention mechanism).
- **Frameworks:** **PyTorch** (Industry standard for AI development).

6. Generative AI (The Modern Era)

- **LLM Concepts:** Tokens, Context Window, Temperature, Top-P.
 - **Model Interaction:** OpenAI API, Anthropic, Google Gemma.
 - **Local AI:** Ollama, LM Studio (Running private brains).
 - **AI Security (The Guardrail Layer):**
 1. **Prompt Injection Defense:** Implementing system prompts that resist "jailbreaking."
 2. **Input Sanitization:** Filtering malicious patterns from user prompts.
 3. **Hallucination Control:** Implementing Citation-based verification (forcing the AI to prove its answer with source data).
 - **Fine-Tuning:** LoRA, QLoRA (Teaching models custom company data).
-

Phase 3: The System (Backend Engineering & RAG)

Goal: Designing the high-scale architecture where AI lives.

7. Full-Stack Backend (FastAPI Suite)

- **Framework:** FastAPI (Async, Pydantic v2 validation).
- **ORM:** SQLAlchemy 2.0 (Async mode).
- **Identity & Access Management (IAM):**
 1. **OAuth2 + JWT:** Secure token-based authentication.
 2. **RBAC (Role-Based Access Control):** Differentiating permissions (User vs. Admin).
 3. **Passlib:** Secure password hashing (Bcrypt).
- **Performance & Protection:**
 1. **Redis:** Caching and Distributed Rate Limiting → Preventing API abuse and protecting LLM budgets.
 2. **Reliability:** Circuit Breakers (Tenacity) and Idempotency Keys.

8. RAG (Retrieval Augmented Generation): Long-term Memory

- **Embeddings:** Text-to-Vector conversion.
- **Vector DBs:** pgvector (Preferred), Pinecone, ChromaDB, or Milvus.
- **Pipeline:** Chunking → Embedding → Storing → Hybrid Search → Re-ranking.
- **Privacy Layer: PII Redaction** → Using tools like Microsoft Presidio to mask sensitive data before it enters the Vector DB.

9. Background Processing

- **Task Queue:** Celery + Redis (For long AI generations).
 - **Monitoring:** Flower (Visualizing background tasks).
-

Phase 4: The Agents & Senses (Autonomous Logic)

Goal: Giving the AI "hands" to perform tasks and "ears/mouth" to speak.

10. AI Agents & Orchestration

- **Agent Logic:** ReAct prompting, **Function Calling** (AI using your Python backend as a tool).
- **Frameworks:** LangChain, LangGraph (Preferred), or CrewAI.
- **Agentic Security:**
 1. **Tool-Access Control:** Restricting which tools the AI can call (e.g., the AI can Read a file but cannot Delete it).
 2. **Human-in-the-Loop (HITL):** Implementing a "Review Step" for critical agent actions.
- **Connectivity:** MCP (Model Context Protocol) for secure local file/tool access.

11. Senses (Speech)

- **STT:** OpenAI Whisper (Hearing).
- **TTS:** ElevenLabs or OpenAI TTS (Speaking).
- **Input Filtering:** Sanitizing audio-to-text transcripts before processing.

Phase 5: The Professional Layer (Observability)

Goal: Deploying, monitoring, and scaling the sandwich.

12. LLMOps & Observability

- **Tracing:** Langfuse or LangSmith (Monitoring every "thought" the AI has to debug hallucinations and latency.)
- **Error Tracking:** Sentry.
- **Metrics:** Prometheus & Grafana (Visualizing system health).

13. Production Deployment (The Fortress)

- **Infrastructure as Code (IaC):** Terraform.
- **Cloud Deployment:** AWS (ECS/Lambda).
- **Network Security:**
 1. **AWS Security Groups:** Closing all ports except 80/443.
 2. **VPC (Virtual Private Cloud):** Isolating the Database and Redis from the public internet.
- **Edge Security:** Caddy or Nginx (SSL/TLS encryption and Reverse Proxy).

The Final Projects

1. **The "Financial Jarvis":**
 - *AI:* A RAG system that reads your bank statements and analyzes spending.
 - *Backend:* **FastAPI** handles user auth; **Celery** processes large PDF uploads; **Redis** limits API usage.
2. **The "Autonomous Coder" Agent:**
 - *AI:* An agent that finds bugs in a repo using **LangGraph**.
 - *Backend:* Uses **Docker** to run the code in a safe "sandbox"; **PostgreSQL** stores the bug history; **MCP** connects to your local VS Code.
3. **Voice-Driven Medical Assistant:**
 - *AI:* **Whisper** hears the patient; **Gemma** diagnoses; **ElevenLabs** speaks back.
 - *Backend:* **SQLAlchemy** stores patient records; **Pydantic** ensures medical data is strictly formatted; **Sentry** alerts if the AI gives a high-risk answer.